

```
1 import static org.junit.Assert.assertEquals;
2
3 import org.junit.Test;
4
5 import components.sequence.Sequence;
6 import components.sequence.Sequence1L;
7
8 /**
9  * Sample JUnit test fixture for SequenceSmooth.
10  *
11  * @author Zhuangzhuang He
12  *
13  */
14 public final class SequenceSmoothTest {
15
16     /**
17      * Constructs and returns a sequence of the integers provided as
18      *
19      * @param args
20      *      0 or more integer arguments
21      * @return the sequence of the given arguments
22      * @ensures createFromArgs= [the sequence of integers in args]
23      */
24     private Sequence<Integer> createFromArgs(Integer... args) {
25         Sequence<Integer> s = new Sequence1L<Integer>();
26         for (Integer x : args) {
27             s.add(s.length(), x);
28         }
29         return s;
30     }
31
32     /**
33      * Test smooth with s1 = <2, 4, 6> and s2 = <-5, 12>.
34      */
35     @Test
36     public void test1() {
37         /*
38          * Set up variables and call method under test
39          */
40         Sequence<Integer> seq1 = this.createFromArgs(2, 4, 6);
41         Sequence<Integer> expectedSeq1 = this.createFromArgs(2, 4, 6);
42         Sequence<Integer> seq2 = this.createFromArgs(-5, 12);
43         Sequence<Integer> expectedSeq2 = this.createFromArgs(3, 5);
44         SequenceSmooth.smooth(seq1, seq2);
45         /*
46          * Assert that values of variables match expectations
47          */
48         assertEquals(expectedSeq1, seq1);
49         assertEquals(expectedSeq2, seq2);
50     }
51
52     /**
```

```
53     * Test smooth with s1 = <7> and s2 = <13, 17, 11>.
54     */
55     @Test
56     public void test2() {
57         /*
58          * Set up variables and call method under test
59          */
60         Sequence<Integer> seq1 = this.createFromArgs(7);
61         Sequence<Integer> expectedSeq1 = this.createFromArgs(7);
62         Sequence<Integer> seq2 = this.createFromArgs(13, 17, 11);
63         Sequence<Integer> expectedSeq2 = this.createFromArgs();
64         SequenceSmooth.smooth(seq1, seq2);
65         /*
66          * Assert that values of variables match expectations
67          */
68         assertEquals(expectedSeq1, seq1);
69         assertEquals(expectedSeq2, seq2);
70     }
71
72     @Test
73     public void testTruncation() {
74         Sequence<Integer> seq1 = this.createFromArgs(2,3,4);
75         Sequence<Integer> expectedSeq1 = this.createFromArgs(2,3,4);
76         Sequence<Integer> seq2 = this.createFromArgs(1,5);
77         Sequence<Integer> expectedSeq2 = this.createFromArgs(2,3);
78         SequenceSmooth.smooth(seq1,seq2);
79
80         assertEquals(expectedSeq1,seq1);
81         assertEquals(expectedSeq2,seq2);
82
83
84     }
85     @Test
86     public void testNegativeNumbers() {
87         /* Math:  $(-10 + -20)/2 = -15$ ,  $(-20 + 40)/2 = 10$  */
88         Sequence<Integer> seq1 = this.createFromArgs(-10, -20, 40);
89         Sequence<Integer> expectedSeq1 = this.createFromArgs(-10, -20, 40);
90         Sequence<Integer> seq2 = this.createFromArgs(0, 0);
91         Sequence<Integer> expectedSeq2 = this.createFromArgs(-15, 10);
92
93         SequenceSmooth.smooth(seq1, seq2);
94
95         assertEquals(expectedSeq1, seq1);
96         assertEquals(expectedSeq2, seq2);
97     }
98
99     @Test
100    public void testLongSequence() {
101        /* s1 length 5, s2 length should be 4 */
102        Sequence<Integer> seq1 = this.createFromArgs(0, 10, 0, 10, 0);
103        Sequence<Integer> expectedSeq1 = this.createFromArgs(0, 10, 0, 10,
104        Sequence<Integer> seq2 = this.createFromArgs(-1, -1, -1, -1);
```

```
105     Sequence<Integer> expectedSeq2 = this.createFromArgs(5, 5, 5, 5);
106
107     SequenceSmooth.smooth(seq1, seq2);
108
109     assertEquals(expectedSeq1, seq1);
110     assertEquals(expectedSeq2, seq2);
111 }
112
113 }
```